



USC University of
Southern California

Prompting Techniques

The Lever

CSCI 599 — Agentic AI · Lecture 4 / Week 2

Prof. Marco Papa

USC



Last lecture, the substrate. Today, the lever.

Substrate: which model lives in the node

- Gemini 3.5 Flash, GPT-5.5, Claude Sonnet 4.6, Gemma 4 — the model layer we evaluated last lecture

Lever: how you talk to whatever model is in front of you

- Same model, different prompt — different output

Today: the lever in five forms. One model. One task.

One model, one task, five techniques

Model: GPT-5.5 — same task via Python API and Codex

Task: code review of a 30-line Python file with deliberate issues

Techniques: zero-shot, few-shot, CoT, configurable thinking, ReAct

Any difference you see is the technique, not the model.

Foundations



The prompt is the agent's API

A chatbot prompt is conversation

- Send words, receive words, vibes matter

An agent prompt is a contract

- Defines tools, output format, constraints, escalation rules
- The contract spans messages, tools, constraints, and output shape. Treat it like an API.

Anatomy of an agent prompt

```
messages = [  
  {"role": "system",  
    "content": "You are a code reviewer. "  
    "Use tools when needed. "  
    "Output as JSON: {issues: [...]}.",  
  {"role": "user",      "content": prior_question},  
  {"role": "assistant", "content": prior_answer},  
  {"role": "tool",      "content": tool_result,  
    "tool_call_id": "..."},  
  {"role": "user",      "content": current_ask},  
]
```

system

Role, policy, behavioral contract

user / assistant

Prior conversation (state)

tool

Tool result (observations)

Tool schemas

Supplied separately by the API/framework

user (last)

Current ask

Generation and reasoning control

Dial	What it controls	Range
temperature	Randomness — lower = more deterministic	0.0 to 2.0
top_p	Nucleus sampling — vocabulary subset	0.0 to 1.0
verbosity	Output length, independent of reasoning	low / medium / high
reasoning_effort	How much native reasoning effort to use	none / low / medium / high / xhigh

Not all controls exist on every model or endpoint. Check current model documentation before relying on a specific control. `reasoning_effort` is the configurable-thinking control we use next.

The five techniques today

Technique	One-line description	Primary use
Zero-shot	Ask, get an answer	Anything the model can handle from the instruction
Few-shot	Show examples first	Style matching, niche formats
Chain-of-thought	Elicit intermediate reasoning steps	Math, multi-step reasoning
Configurable thinking	Control native reasoning effort	Hard tasks; trade compute for depth
ReAct	Reason → act → observe → repeat	Tool-using agents

Each gets its own slide. Then the demo runs all five.

The Five Techniques



Zero-shot: the baseline

Ask, get an answer. No examples, no scaffolding.

Works when the task and desired output are clear from the instruction

- Common code patterns, well-known concepts, standard formats

Fails on: ambiguous tasks, niche formats, domain-specific output

The honest first attempt before reaching for anything fancier.

Few-shot: show before you tell

Brown et al., 2020 — arXiv:2005.14165 (the GPT-3 paper)

Include a small set of representative input/output examples before the actual ask

- Model picks up the pattern from the examples
- Especially powerful for style, format, niche conventions

Cost: every example burns context tokens

PAPA'S RULE

Few-shot only when you can name what zero-shot missed.

Few-shot — in code

```
from openai import OpenAI
client = OpenAI()

messages = [
    {"role": "system",
     "content": "Review Python code. Output one issue per line."},
    # Example 1
    {"role": "user", "content": "def f(x): return x/0"},
    {"role": "assistant",
     "content": "L1: ZeroDivisionError – guard against x == 0"},
    # Example 2
    {"role": "user", "content": "eval(user_input)"},
    {"role": "assistant",
     "content": "L1: eval on user input – arbitrary code execution"},
    # Actual ask
    {"role": "user", "content": code_to_review},
]
response = client.chat.completions.create(
    model="gpt-5.5", messages=messages, reasoning_effort="medium")
```

Chain-of-thought: elicit intermediate reasoning

Wei et al., 2022 — arXiv:2201.11903

Prompting can elicit intermediate reasoning steps before an answer

- Few-shot CoT: examples include the reasoning, not just answers
- Zero-shot CoT: Kojima et al. 2022 — arXiv:2205.11916
 - Let's think step by step — a landmark zero-shot CoT result

Helps on: arithmetic, logical chains, multi-step deductions

CoT — in code

```
from openai import OpenAI
client = OpenAI()

# Zero-shot CoT — landmark result (Kojima 2022)
response = client.chat.completions.create(
    model="gpt-5.5",
    messages=[
        {"role": "system",
         "content": "Review Python code. "
                    "Let's think step by step. "
                    "For each line, ask: what could go wrong here? "
                    "Then list the issues found."},
        {"role": "user", "content": code_to_review},
    ],
    reasoning_effort="low", # hold native reasoning effort low for this comparison
)
```

Configurable thinking: dial the model up

Tell the model how much native reasoning effort to use

- CoT is prompted. Configurable thinking changes native reasoning effort.
- Same prompt, more reasoning effort — same task, more compute

Two mechanisms, different control surfaces

- GPT-5.5: reasoning_effort parameter (none / low / medium / high / xhigh)
- Gemma 4: thinking mode via the [think] control token

Native in newer models; availability varies by model and endpoint

GPT-5.5: reasoning_effort

```
# Same prompt, two settings, different outputs
response_quick = client.chat.completions.create(
    model="gpt-5.5",
    messages=[{"role": "user", "content": code_to_review}],
    reasoning_effort="low",          # lower native reasoning effort
)

response_deep = client.chat.completions.create(
    model="gpt-5.5",
    messages=[{"role": "user", "content": code_to_review}],
    reasoning_effort="xhigh",       # higher native reasoning effort
)

# Measure the quality / latency tradeoff on the actual workload.
```

PAPA'S RULE

Reach for reasoning_effort before reaching for the next-tier model.

Gemma 4: the `|think|` token returns

```
# Same technique, different mechanism (from Lecture 3)
from langchain_ollama import ChatOllama

llm = ChatOllama(model="gemma4:e4b")
response = llm.invoke([
    {"role": "system", "content": "<|think|>"}, # enable thinking
    {"role": "user", "content": code_to_review},
])
# This model family exposes thinking through a control token,
# not the same parameter surface used by GPT-5.5.
```

One technique. Two mechanisms. The implementation differs by model family — the lesson is the same.

ReAct: reason and act

Yao et al., 2022 — arXiv:2210.03629 (ICLR 2023)

Interleave thought, action, observation in one loop

- Thought — what the model thinks should happen next
- Action — a tool call the model emits
- Observation — the tool result fed back to the model

A foundational pattern for many modern tool-using agents — explicit or abstracted behind a framework.

ReAct — the loop as text

Thought: I need to see what the file imports
to understand the scope.

Action: `read_file(path="auth.py", lines="1-10")`

Observation: `import sqlite3; import hashlib; ...`

Thought: It uses raw `sqlite3` – check for SQL
injection in query construction.

Action: `grep_file(path="auth.py", pattern="execute")`

Observation: `cursor.execute(f"SELECT * FROM users
WHERE name='{name}')`

Thought: That's string interpolation into SQL –
classic injection vector. Flag it.

Week 3 builds this loop from scratch. Today we use the canonical ReAct abstraction; production agents may keep reasoning hidden.

The Demo



Demo: review this code, five ways

```
# sample_code.py — the file under review
def authenticate(username, password):
    conn = sqlite3.connect("users.db")
    cursor = conn.cursor()
    cursor.execute(
        f"SELECT * FROM users WHERE name='{username}'"
    )
    row = cursor.fetchone()
    if row and row[2] == hashlib.md5(
        password.encode()).hexdigest():
        return True
    return False

# Three deliberate issues. Run five techniques.
```

What to watch

- Python API sweep — one measured run
- Latency per technique
- Issues found
- Output structure
- Which issues each technique misses

What the demo showed

Technique	Wall-clock	Issues found	Output style
Zero-shot	26.79s	11 total; SQL ✓, MD5 ✓, missed no-validation	Numbered markdown, per issue code fixes
Few-shot	17.71s	5 total; SQL ✓, MD5 ✓, missed no-validation	Terse one-liners: line range, issue, fix
CoT (zero-shot)	38.09s	15 total; all 3 deliberate ✓	Line-by-line walkthrough, then summary
reasoning_effort=xhigh	155.72s (~2.5 min)	5 findings + schema/production guide	Structured audit + full migration path
ReAct (+ tools)	15.27s / 4 tool calls	4 findings; all 3 ✓, cites CWE-89	Trace-driven with risk + CWE categorization

Python API sweep — one measured run

Output shape differs across techniques

Few-shot inherits the examples' format AND scope — verbatim

CoT walks line by line — catches abstract issues concrete techniques miss

Configurable thinking produces an audit report — schema, migration, production

ReAct produces a tool-informed trace — observations come from the available tool environment

The format of your prompt influences both what the model looks for and how it reports findings.

When each technique misfires

Zero-shot on ambiguous tasks — confident wrong answers

Few-shot when examples bias the answer — pattern-mimicking $\neq$ thinking

CoT on tasks that don't need reasoning — adds tokens/latency without measurable benefit

Configurable thinking without enough signal — wastes compute

ReAct without good tools — can waste iterations or cycle until a configured limit

Cheat sheet: which technique, when

Task shape	First reach for	If that fails
Standard task, common pattern	Zero-shot	Few-shot with 2-3 examples
Niche format or domain	Few-shot	Few-shot + CoT in examples
Multi-step reasoning	CoT (zero-shot)	reasoning_effort=medium or high
Deep single-answer problem	reasoning_effort=xhigh	Accept the latency cost
Needs external information	ReAct with tools	Better tools, not bigger model

Composition is real - techniques layer.

Next: planning and reasoning loops

Week 3: planning agents, reasoning loops, ReAct in depth

- Build the loop from scratch — no framework
- Self-consistency, tree-of-thought, plan-and-execute
- When loops fail: cycles, infinite tools, lost context

Today's techniques compose into multi-step agents.